Aran Learn 5: AI and NLP for Detection

For: Bharadhwaj and Arun, who own Aran's AI part; others read sections 1 and 2. **Time:** about 8 hours with practice. **Before this:** Learn 1 and Learn 4 sections 1 to 3.

Checksums catch Aadhaar and card numbers, but names, addresses and OTPs have no checksum. For those, Aran learns from examples. This doc takes you from "what is machine learning" to a working name finder and a calibrated OTP classifier, with code you can run on a laptop CPU.

1. Machine learning basics

1.1 The idea

Normal programming: you write the rule, the computer applies it. Machine learning: you give the computer examples with the right answers, and it finds a rule that reproduces them. Aran uses it only where hand-written rules fail.

Problem	Why a hand-written rule fails	What ML learns instead
Is "Madurai" a place in this log line?	No list of all Indian place names is complete, and names are spelled many ways	Clues such as "word after 'in'", "ends in -pur, -nagar, -abad"
Is "Ramesh Kumar" a person?	Same; and "Kumar" alone can be part of a company name	Capitalisation, neighbouring words, titles like "Mr", "Smt"
Is 482913 an OTP or a port number?	Both are 6 digits	The surrounding words: "your OTP is" against "port ... refused"

1.2 The vocabulary

Term	Meaning	Aran example
Example (sample)	One item to classify	One log line, or one word
Label	The right answer for an example	OTP or not; B-PER, I-PER, O
Feature	A measurable clue about an example	"word is capitalised", "previous word is Mr"
Model	The learned rule mapping features to an answer	Logistic regression, CRF, transformer
Training	Adjusting the model's numbers so its answers match the labels	`model.fit(X_train, y_train)`
Test set	Examples held back and never shown during training	The CoNLL-2003 test file
Overfitting	Memorising the training examples instead of the pattern	99% on training data, 60% on test data

1.3 The golden rule: never test on training data

A student who has seen the exam paper scores 100% and has learned nothing. A model is the same. Always split first, train on one part, and report results only on the other.

```
        from sklearn.model_selection import train_test_split
        X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0,
stratify=y)
```

`stratify=y` keeps the same share of each label in both parts, which matters when one label is rare (OTPs are rare in logs). `random_state=0` makes the split repeatable, so teammates get the same numbers.

A subtler trap: template leakage. If you generate training sentences from templates ("your OTP is {n}", "{n} is your code"), a random split puts the same template in both parts, and the model looks great just by recognising templates. Split by template instead: some templates only in training, others only in testing. Then the test checks whether the model handles wordings it has never seen, which is what real bank messages will throw at it.

1.4 Logistic regression, the first model to learn

Logistic regression gives each feature a weight, adds them up with a bias, and squashes the total into a probability between 0 and 1 using the sigmoid function, σ(z) = 1 / (1 + e^(−z)).

Worked example with three hand-picked features for "is this number an OTP?":

Feature	Weight
bias	−4
text contains "otp" or "code"	+5
text contains "share"	+2
text contains "blk_" or "port"	−3

Line	z	σ(z)
"your otp is 482913"	−4 + 5 = 1	0.73
"do not share otp 482913"	−4 + 5 + 2 = 3	0.95
"listening on port 482913"	−4 − 3 = −7	0.001
"482913" alone	−4	0.018

Training finds these weights automatically from labelled lines, for thousands of features at once. The model is easy to inspect: sort the weights and you see which clues it relies on, which helps when a bank's auditor asks why something was flagged.

2. Measuring a model

Learn 4 section 2 introduced precision, recall and F1. For AI models two more ideas matter.

Entity-level scoring. For names, a prediction counts as correct only if the whole entity and its type match. If the text says "Ramesh Kumar" and the model tags only "Ramesh", that is one miss and one wrong prediction, not "half right".

Worked example, gold entities {Ramesh Kumar: PER, Madurai: LOC, SBI: ORG}, predictions {Ramesh: PER, Madurai: LOC, SBI: ORG, Kumar: PER}:

- Correct: Madurai, SBI, so TP = 2.
- Predicted 4, so precision = 2 / 4 = 50%.
- Gold 3, so recall = 2 / 3 = 67%.
- F1 = 2 × 0.5 × 0.67 / (0.5 + 0.67) ≈ 0.57.

Report per type. Always report F1 separately for PER and LOC. A model can find names well and still miss addresses, and for Aran a missed address is a leak.

3. Named entity recognition (NER)

3.1 BIO tagging

NER finds names of people, places and organisations in text. It is framed as tagging every word: B (beginning of an entity), I (inside, continuing it) or O (outside any entity).

Word	Ramesh	Kumar	from	Anna	Nagar	paid	SBI
Tag	B-PER	I-PER	O	B-LOC	I-LOC	O	B-ORG

Why B and I and not just PER? Because two names can sit next to each other ("gave Ramesh Suresh's card"), and B marks where the second one starts.

3.2 Features for each word

A CRF does not read words; it reads features you compute for each word. A typical feature function:

```
        def word2features(sent, i):
            w = sent[i]
            f = {
                "bias": 1.0,
                "lower": w.lower(),
                "suffix3": w[-3:].lower(),        # '-pur', '-bad', '-esh'
                "is_title": w.istitle(),          # 'Ramesh'
                "is_upper": w.isupper(),          # 'SBI'
                "is_digit": w.isdigit(),
            }
            if i > 0:
                f["prev_lower"] = sent[i - 1].lower()   # 'mr', 'in', 'from'
            else:
                f["BOS"] = True                        # beginning of sentence
            if i < len(sent) - 1:
                f["next_lower"] = sent[i + 1].lower()
            else:
                f["EOS"] = True
            return f

        def sent2features(sent):
            return [word2features(sent, i) for i in range(len(sent))]
```

3.3 Conditional random fields (CRF)

A CRF looks at each word's features and also at the tag of the word before it. So it learns rules such as "I-PER can only follow B-PER or I-PER" and "after 'in', a capitalised word is likely B-LOC". It trains in minutes on a laptop CPU, and its learned weights can be printed and explained, which is why we start with it.

```
        import sklearn_crfsuite
        from sklearn_crfsuite import metrics

        X_train = [sent2features(s) for s in train_sents]   # list of sentences, each a
list of words
        y_train = train_tags                                 # matching lists of BIO tags

        crf = sklearn_crfsuite.CRF(algorithm="lbfgs", c1=0.1, c2=0.1,
                                   max_iterations=100, all_possible_transitions=True)
        crf.fit(X_train, y_train)

        y_pred = crf.predict([sent2features(s) for s in test_sents])
        labels = [l for l in crf.classes_ if l != "O"]
        print(metrics.flat_classification_report(test_tags, y_pred, labels=labels,
digits=3))
```

`c1` and `c2` are regularisation strengths: they stop the model from leaning too hard on rare features, which is the main defence against overfitting. Try values from 0.01 to 1 and keep the one that scores best on a separate validation split, never on the test set.

`flat_classification_report` scores word by word. For the entity-level score of section 2, use the `seqeval` library on the same lists.

3.4 Transformers, the next step

A transformer such as BERT turns each word piece into a vector of numbers (an embedding) and then lets every word "attend" to every other word in the sentence, so the meaning of "Nagar" can depend on "Anna" two words before. Models pre-trained on huge text collections already know a lot about language; you only **fine-tune** them on your labelled data.

Model	Trained on	Why it suits Aran
MuRIL (Google)	17 Indian languages plus transliterated text	Handles Hinglish and Tanglish in Roman script
IndicBERT (AI4Bharat)	12 Indian languages	Small and fast
XLM-RoBERTa	100 languages	Strong general baseline

They usually beat a CRF by several F1 points but need a GPU for reasonable training time (Google Colab's free GPU is enough for our datasets). Plan: build the CRF baseline first, then fine-tune MuRIL with Hugging Face `transformers` and report both on the same test set. A better model only counts if it beats the baseline on identical data.

3.5 The datasets

Dataset	Language	What it is	Where
CoNLL-2003	English	Reuters news, about 300,000 tagged words; the standard NER benchmark	GitHub mirror (synalp/NER)
Singh et al. 2018, Hindi-English code-mixed NER	Hinglish (Roman script)	Real tweets, about 72,000 tagged words; Person, Location, Organisation	GitHub (SilentFlame/Named-Entity-Recognition)
HiNER (LREC 2022)	Hindi (Devanagari)	About 100,000 sentences	Hugging Face (cfilt/HiNER-original); download on your own laptop

None of these is bank data, and that is deliberate: we must not train on real customers' messages. For bank-style text, write templates with invented names from public name lists and test on held-out templates (section 1.3).

4. The OTP context classifier

4.1 The problem in numbers

On 32,000 real log lines with no OTPs at all, a plain 6-digit regex fired 67,481 times. A keyword rule cut that to 6, but keyword lists miss new wordings ("aapka code", "kisi ko na batayen"). A classifier learns the clues from examples instead.

4.2 Character n-grams

Break the text around the number into overlapping pieces of 2 to 5 characters. "otp is" becomes "ot", "tp", "p ", " i", "is", "otp", "tp ", ... This copes with spelling variety, typos and Hinglish without any dictionary, because "OTP", "otp:" and "otp-" share most of their pieces.

Replace the number itself with a placeholder such as `<NUM>` before training. Otherwise the model may learn that particular digits mean OTP, which is meaningless, and you would be storing the numbers in the training data.

4.3 A complete small example

```
        from sklearn.feature_extraction.text import TfidfVectorizer
        from sklearn.linear_model import LogisticRegression
        from sklearn.pipeline import make_pipeline

        texts = [
            "your otp is <NUM> do not share", "aapka OTP <NUM> hai", "<NUM> is your
verification code",
            "use passcode <NUM> to login", "OTP for txn of Rs 500 is <NUM>", "never share
otp <NUM> with anyone",
            "one time password <NUM> valid 5 min", "login code <NUM> expires soon",
            "blk_<NUM> replicated to 3 nodes", "took <NUM> ms to respond", "port <NUM>
connection refused",
            "pid <NUM> exited with status 0", "received <NUM> bytes from peer", "session
<NUM> closed by user",
            "queue size <NUM> exceeds limit", "heap usage <NUM> kb",
        ]
        y = [1] * 8 + [0] * 8

        clf = make_pipeline(
            TfidfVectorizer(analyzer="char_wb", ngram_range=(2, 5)),
            LogisticRegression(max_iter=1000, C=10),
        )
        clf.fit(texts, y)

        for t in ["aapka otp <NUM> hai, kisi ko na batayen",
                  "fetched <NUM> bytes in block",
                  "verification code is <NUM>"]:
            print(t, round(clf.predict_proba([t])[0][1], 2))
        # aapka otp <NUM> hai, kisi ko na batayen 0.78
        # fetched <NUM> bytes in block 0.26
        # verification code is <NUM> 0.79
```

With only 16 examples the scores are modest, but the direction is already right, including on a Hinglish sentence it never saw. With a few thousand varied examples the separation becomes sharp. `TfidfVectorizer` gives each n-gram a weight that is high when it is frequent in this line but rare overall; `char_wb` keeps n-grams inside word boundaries.

5. Calibration: making probabilities honest

5.1 Why it matters for Aran

Learn 4 combines evidence by multiplying likelihood ratios, and that only works if each probability means what it says. A model that outputs 0.9 should be right about 90% of the time. Raw model scores often are not: they can be overconfident (0.99 when right 80% of the time) or timid.

5.2 Reliability diagram

Group predictions into bins by score (0 to 0.1, 0.1 to 0.2, ...). For each bin, plot the average predicted probability against the actual fraction that were positive. A perfectly calibrated model lies on the diagonal.

Bin	Average predicted	Actual positive rate	Verdict
0.8 to 0.9	0.85	0.62	Overconfident
0.9 to 1.0	0.97	0.81	Overconfident

5.3 Fixing it

Isotonic regression learns a mapping from raw score to true frequency that only ever goes up, using held-out data the model did not train on. scikit-learn wraps it:

```
from sklearn.calibration import CalibratedClassifierCV
cal = CalibratedClassifierCV(clf, method="isotonic", cv=5).fit(X_tr, y_tr)
p = cal.predict_proba(X_te)[:, 1]
```

Isotonic needs a fair amount of data (several hundred examples per class). With less, use `method="sigmoid"` (Platt scaling).

5.4 Metrics for probabilities

Metric	What it tells you	Better is
Precision, recall, F1	Quality of the yes or no decision at one threshold	Higher
ROC AUC	How well scores rank real above fake, over all thresholds	Higher (1.0 perfect, 0.5 random)
Brier score	Average squared gap between probability and outcome	Lower
Expected calibration error (ECE)	Average gap between predicted probability and actual frequency, weighted by bin size	Lower

Brier worked example. Predictions 0.9, 0.8, 0.3, 0.1 for outcomes 1, 1, 0, 0. Squared errors: 0.01, 0.04, 0.09, 0.01. Brier = 0.15 / 4 = 0.0375.

6. Responsible AI in a bank

- **No real customer data in training sets or notebooks.** Use public datasets and invented examples. A model can memorise and repeat training text.
- **Explain decisions.** Keep the CRF and logistic regression baselines because their weights can be shown to an auditor.
- **Fail safe.** If the model is unsure on a value near a sensitive context, mask it. A wrongly masked port number costs little; a leaked OTP costs a customer's money.
- **Watch for drift.** New SMS templates and new log formats appear over time. Re-measure monthly on fresh labelled samples.

7. Practice

- **P1.** Install `sklearn-crfsuite`, load the first 2,000 sentences of CoNLL-2003, train the CRF of section 3.3, and report the F1 for PER and LOC.
- **P2.** Add one feature (for example `suffix2`, or "word is all capitals") and see whether F1 changes. Keep a table of every experiment.

- [ ] **P3.** Run the OTP example of section 4.3. Then each of you writes 10 new test sentences the other has not seen and measures accuracy.
- [ ] **P4.** Rebuild the OTP data with 10 templates per class, split by template, and compare accuracy with a random split. Explain the difference.
- [ ] **P5.** Plot a reliability diagram for your OTP model before and after `CalibratedClassifierCV`, and report Brier score and ECE for both.
- [ ] **P6.** Print the 10 largest positive and negative weights of the OTP logistic regression (`clf[-1].coef_` with `clf[0].get_feature_names_out()`) and explain them.

8. Further reading

- Jurafsky and Martin, "Speech and Language Processing" (free online): chapters 4 (classification), 5 (logistic regression), 8 (sequence labelling) and the transformer chapters.
- The scikit-learn user guide sections on model evaluation and probability calibration.
- The Hugging Face course (huggingface.co/learn), chapters 1 to 3, then the token-classification chapter.
- Singh et al. 2018, "Named Entity Recognition for Hindi-English Code-Mixed Social Media Text" (ACL Anthology W18-2405).
- Khanuja et al. 2021, "MuRIL: Multilingual Representations for Indian Languages".